Software de comunicaciones

Curso 2026/2027

Práctica 2

Distribución usando RMI

Indice

Objetivos 1
Entrega de resultados 1
Enunciado 1
Fase 1 3
Fase 2 3
Fase 3 4
Anexo 1 6
Anexo 2 7

Objetivos

- Entender los fundamentos de la distribución de aplicaciones utilizando middleware.
- Entender y aplicar conceptos relativos a la distribución de objetos en general y de RMI de Java en particular.
- Desarrollar una aplicación distribuida utilizando invocación remota de métodos de objetos remotos (RMI) usando RMI de Java.

Entrega de resultados

Se debe entregar un fichero zip en la tarea correspondiente de Moodle con los siguientes contenidos:

1. Carpetas completas de TODOS los proyectos Eclipse creados en la realización de la práctica. Todas las carpetas entregadas deben contener todos los ficheros fuente utilizados.

La entrega en Moodle debe realizarse ANTES del 3 de noviembre.

Los resultados entregados para esta práctica podrán ser necesarios para la realización del examen práctico del primer parcial de la asignatura.

Enunciado

Se dispone de una aplicación que implementa una calculadora con memoria del mismo modo que en la práctica 1. Esta aplicación tiene un funcionamiento monolítico cuyo diagrama de clases se muestra en el Anexo 1 del enunciado de dicha práctica.

La aplicación calculadora dispone de las cuatro operaciones aritméticas básicas, así como de la capacidad de mostrar el último resultado obtenido y de utilizar una memoria acumuladora. La calculadora dispone de una interfaz gráfica de usuario (Graphical User Interface, GUI) que se presenta a continuación.



La calculadora dispone de dos áreas diferenciadas: la de presentación de resultados en la parte superior y los botones en la parte inferior. El área de representación dispone de dos líneas: una para visualizar los resultados de las operaciones y una segunda línea para la visualización de los números que se componen y los resultados obtenidos. Los botones, por su parte, son de tres clases: los que permiten componer un número dígito a dígito, los operadores aritméticos y los de manipulación de las memorias. Las dos primeras clases permiten una operativa evidente, así que no se presenta su funcionalidad. En cuanto a los botones de manipulación de memorias, esta es su funcionalidad:

- **Último resultado (UR)**: Permite visualizar el resultado de la última operación aritmética realizada.
- **Memoria-Añadir (MA)**: Permite añadir a la memoria acumuladora el resultado de la última operación aritmética realizada.
- **Memoria-Obtener (MO)**: Permite visualizar el contenido de la memoria acumuladora.
- **Memoria-Limpiar (ML)**: Permite poner a cero el contenido de la memoria acumuladora.

Existe un botón adicional **Borrar Entrada (BE)**, que permite borrar la entrada actual en caso de cometer un error al componer un número.

Esta práctica se debe realizar siguiendo en secuencia cada una de sus fases.

Antes de comenzar la práctica, el alumno debe disponer de las clases que permitan ofrecer la funcionalidad completa de la aplicación tal y como se ha hecho en fase 1 de la práctica anterior, teniendo especial cuidado en la implementación de la clase **`OperacionesCalculadora`**.

En la primera fase el alumno deberá definir la interfaz de un objeto remoto que implementa las funcionalidades de la interfaz de **`OperacionesCalculadora`**. Esta interfaz se debe denominar **`ICalculadoraRMI`**.

Con posterioridad, en la fase 2 se realizará una distribución de la aplicación utilizando un objeto remoto transitorio con la capacidad de realizar las operaciones, así como la gestión de las memorias de la calculadora, que pasará a ser alojado en un servidor específico. Además, se plantean **dos preguntas de respuesta obligatoria**.

La fase 3 tiene por objeto la protección del acceso concurrente a las variables miembro de la implementación de un objeto remoto tal y como ya se hizo en la práctica anterior. Se plantea además una pregunta de respuesta opcional.

Fase 1

En esta fase, se debe codificar completamente la interfaz **`ICalculadoraRMI`** que debe extender **`java.rmi.Remote`**. El método que implementa la división y la raíz cuadrada deben arrojar una excepción de usuario de comprobación obligatoria de tipo **`CalculadoraExcepcion`** que tenga como superclase **`java.lang.Exception`** y que tenga un `serialVersionUID` definido explícito. Debe permitir el transporte de un mensaje textual aportado al constructor que se pueda recuperar posteriormente con el método de la superclase `getMessage()`.

Fase 2

Se debe usar un proyecto Maven (se puede usar una copia del proyecto del servidor de la práctica anterior) de nombre **`p2-servidor`** que incluya los siguientes elementos:

1. Clases del paquete **`calculadora`**:
    a. **`OperacionesCalculadora`**: clase que contiene la implementación original de las operaciones de la calculadora local. Se puede reutilizar la implementación realizada en la práctica anterior si ha pasado todos los tests.
2. Clases del paquete **`rmiCalculadora`**:
    a. **`ICalculadoraRMI`**: interfaz definida en la Fase 1.
    b. **`CalculadoraExcepcion`**: clase definida en la Fase 1.
3. Clases del paquete **`servidor`**:
    a. **`ServidorCalculadora`**: implementa el modelo general de un servidor que registra y sirve un objeto remoto.
    b. **`AdaptadorOperacionesCalculadoraRMI`**: clase que implementa el adaptador a la interfaz del objeto remoto y que utiliza la implementación de **`OperacionesCalculadora`** como clase sirviente (servant).

Se debe usar también un proyecto Maven (se puede usar una copia del proyecto del cliente de la práctica anterior) de nombre **`p2-cliente`** que incluya los siguientes elementos:

1. Clases del paquete **`cliente`**:
    a. **`ClienteCalculadoraGUI`**: clase que contiene el método **`main()`**. Su implementación debe tener la misma implementación que la usada para la calculadora local.
    b. **`AdaptadorOperacionesCalculadoraGUI`**: adaptador de objeto que se debe encargar de la obtención del objeto remoto, la invocación de sus métodos, la gestión de excepciones y su relanzamiento en caso necesario.
2. Clases del paquete **`rmiCalculadora`**:
    a. **`ICalculadoraRMI`**: interfaz definida en la Fase 1.
    b. **`CalculadoraExcepcion`**: Excepción de usuario subclase de **`java.lang.Exception`**. Debe permitir el transporte de un mensaje textual aportado al constructor que se pueda recuperar posteriormente con el método de la superclase `getMessage()`. Debe tratarse de la misma clase usada en el proyecto del servidor y tener exactamente el mismo valor `serialVersionUID` definido explícito que en el servidor.
3. Paquete **`CalculadoraGUI`** contenido en el archivo **`CalculadoraGUI.jar`** que se ofrece ya implementado.

Es necesario realizar las pruebas oportunas para verificar que las implementaciones de las clases son coherentes con las funcionalidades requeridas para la calculadora. No se debe continuar con la siguiente fase hasta que no se haya terminado completamente esta.

NOTAS:

a. No se requiere de utilizar ninguna dependencia (más allá del paquete **`CalculadoraGUI`** que necesita el cliente) en los proyectos dado que RMI pertenece al API de Java. Si se reutilizan los proyectos de la práctica anterior, es necesario revisar las dependencias y plugins y eliminar lo relativo a protobuf y gGRPC.
b. Cada proyecto Maven (**`p2-servidor`** y **`p2-cliente`**) debe tener un **`artifactId`** en **`pom.xml`** que coincida con el nombre de dicho proyecto.

PREGUNTAS 1 y 2: ¿Qué observa si en un cliente realiza una operación y en otro cliente simultáneo se solicita visualizar el último resultado? ¿Por qué ocurre eso?

Se deben incluir las respuestas a estas preguntas como comentarios iniciales del archivo ServidorCalculadora.java.

Fase 3

1. Modifique la implementación de los métodos de la clase **`OperacionesCalculadora`** para que el acceso concurrente por parte de los diferentes hilos a la única instancia del objeto manejador sea ordenado. Se sugiere considerar el problema como un problema de lectores-escritores y usar la clase **`ReentrantReadWriteLock`** para solucionar el problema.

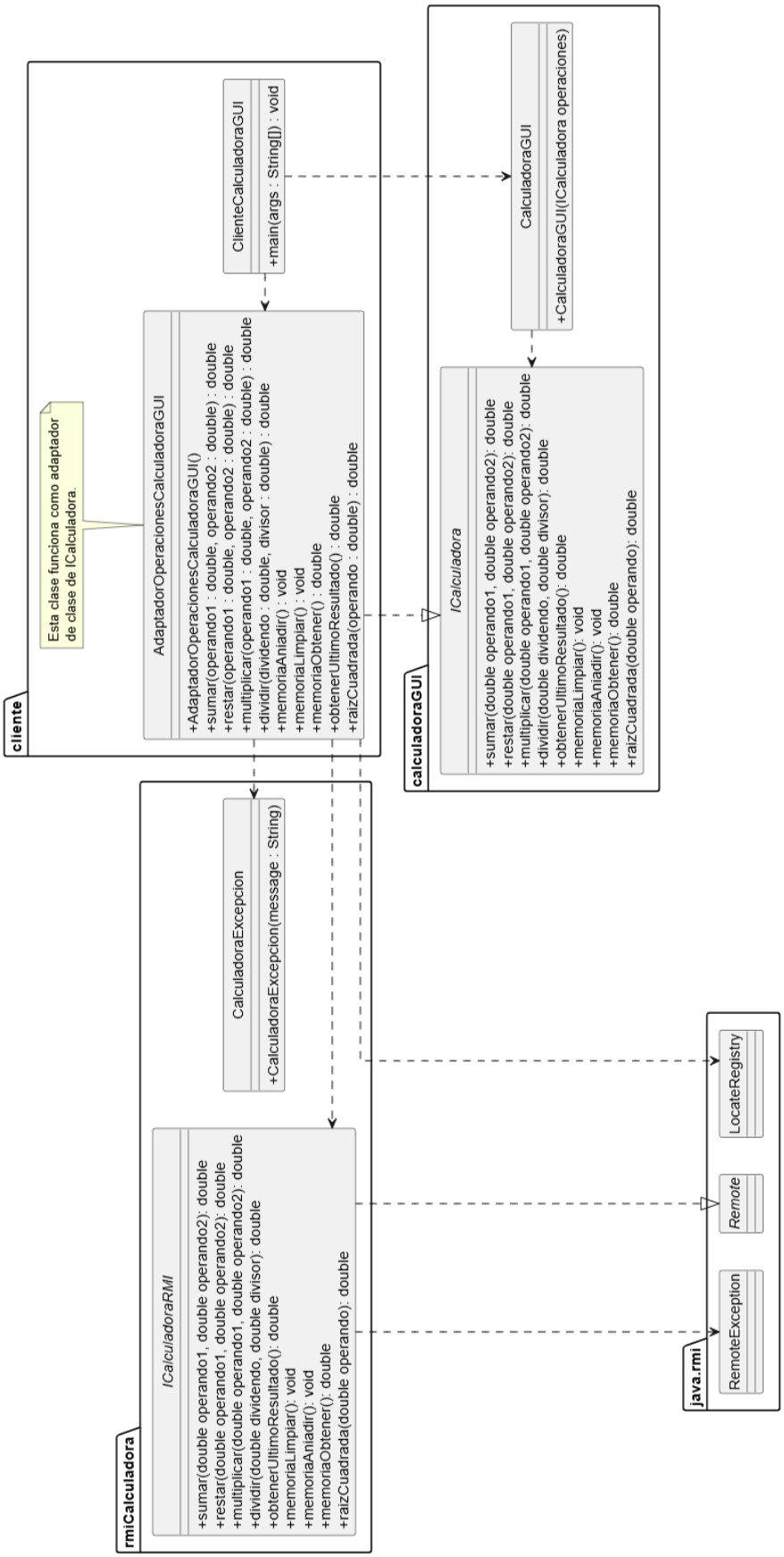
2. **OPCIONAL**

PREGUNTA 3: Si se desea garantizar que cada cliente dispone de una calculadora personal, explique cómo se puede aplicar la técnica de un patrón factoría como el de la referencia oficial de Java para conseguirlo.

Se deben incluir la respuesta a esta pregunta como comentarios iniciales del archivo ServidorCalculadora.java.

Anexo 1

Diagrama de clases de la aplicación cliente calculadora con memoria.



Anexo 2

Diagrama de clases de la aplicación servidor calculadora con memoria.

